

Prompt Coupling: Formalizing Cross-Model Prompt Dependencies in Large Language Model Systems

Abhishek Sharma
Independent Researcher
abhishek@fordelstudios.com

Abstract

Large language model prompts are widely treated as portable text, yet research demonstrates that formatting alone causes accuracy variations of up to 78.3 percentage points within a single model and optimal format overlap between model families falls below an Intersection-over-Union of 0.2. We argue that existing terminology—model drifting, prompt brittleness, prompt sensitivity—describes symptoms of a deeper structural phenomenon we term **prompt coupling**: the degree of interdependence between a prompt and the specific LLM it targets. This is a conceptual and analytical paper. Drawing on five decades of software engineering coupling theory [Stevens et al., 1974, Yourdon and Constantine, 1979], we propose a conceptual framework for prompt coupling with a taxonomy adapted from classical coupling types, introduce the *intent-output pipeline* as a framework for analyzing where coupling enters LLM-based systems, and present systematic observations of coupling patterns in production agentic tools—including 313 model-specific configurations in Aider’s codebase. A survey of 10+ prompt optimization tools confirms that no existing tool performs format-level cross-model compilation. We introduce `promptc`, an open-source transparent HTTP proxy that demonstrates the feasibility of format-level prompt compilation via deterministic structural transforms and optional LLM-powered semantic adaptation.¹ As multi-model deployment strategies become the enterprise norm, prompt coupling represents an unaddressed dimension of vendor lock-in with economic and regulatory implications.

¹<https://github.com/shakecodeslikecray/promptc>

1 Introduction

Consider two prompts for the same classification task. The first uses the format `passage: {} \n answer: {}`. The second removes the colons: `passage {} \n answer {}`. On LLaMA-2-13B, this change—semantically meaningless—causes a swing from 4.3% to 82.6% accuracy: a 78.3 percentage point spread [Sclar et al., 2024]. On GPT-4-32k, switching a prompt from JSON to plain text format yields a 300% performance improvement on the HumanEval benchmark [He et al., 2024].

These are not edge cases. Across 53 tasks from SuperNaturalInstructions, 20% exhibit format-induced spreads of 15 percentage points or more [Sclar et al., 2024]. Optimal prompt formats between model families show Intersection-over-Union scores below 0.2, meaning the formats that work best for one model rarely work best for another [He et al., 2024]. In code generation, prompt perturbations degrade accuracy by up to 21.2% [Chen et al., 2024].

The research community has recognized this phenomenon under several names. Wang et al. [2025] call it “model drifting.” Others describe “prompt brittleness” or “prompt sensitivity” [Ngweta et al., 2025, Razavi et al., 2025]. Each term captures a fragment of the problem. None captures the structural relationship between prompts and models, provides a prescriptive design principle, or connects to the extensive body of work on modularity and dependencies in software engineering.

We propose the term **prompt coupling**, defined as the degree of interdependence between a prompt and the specific LLM implementation it targets. The term borrows directly from Larry Constantine’s foundational work on coupling in software design [Stevens

et al., 1974], where coupling measures interdependence between modules—and where reducing coupling is a fundamental design goal.

This paper makes the following contributions:

1. A **conceptual framework** for prompt coupling with a taxonomy adapted from classical software engineering coupling types.
2. The **intent-output pipeline**, a framework for analyzing where coupling enters LLM-based systems, with particular attention to how agentic tools multiply coupling across layers.
3. A **systematic observation** of coupling patterns in production agentic tools, including Aider’s 313 model-specific configurations, Claude Code’s architectural dependencies, and Cursor’s behavioral assumptions.
4. A **gap analysis** of 10+ prompt optimization tools confirming that no existing tool performs format-level cross-model compilation.
5. **promptc**, an open-source reference implementation demonstrating the feasibility of format-level prompt compilation as a transparent HTTP proxy.

2 Background and Related Work

2.1 Coupling in Software Engineering

The concept of coupling was introduced by Stevens, Myers, and Constantine in their 1974 paper “Structured Design” [Stevens et al., 1974], which generated more reprint requests than any other paper in the IBM Systems Journal’s history. Yourdon and Constantine formalized the concept in their 1979 book [Yourdon and Constantine, 1979], establishing coupling and cohesion as fundamental software engineering principles.

Coupling measures the degree of interdependence between software modules. The classical taxonomy ranges from tight to loose: *content coupling* (one module depends on another’s internal implementation), *common coupling* (modules share global state), *control coupling* (one module controls another’s flow), *stamp coupling* (modules share composite data structures), and *data coupling* (modules communicate only through parameters). Low coupling is a design goal because it enables independent modification, testing, and replacement of modules.

This framework applies directly to the relationship between prompts and language models. A prompt tightly coupled to a specific model’s formatting preferences is analogous to a module that depends on another module’s internal implementation—both create fragile, non-portable dependencies.

2.2 Prompt Format Sensitivity

Sclar et al. [2024] introduced the FormatSpread algorithm to measure performance variation across prompt formats. Testing on LLaMA-2 (7B, 13B, 70B), Falcon-7B, and GPT-3.5-Turbo across 53 tasks, they found a median spread of 7.5 accuracy points, with 20% of tasks exhibiting spreads of 15 points or more. The maximum observed spread was 78.3 percentage points on LLaMA-2-13B. Critically, sensitivity persists with increased model size (LLaMA-2-70B median spread: 0.171) and instruction tuning (Falcon-7B-Instruct shows reduced but non-zero sensitivity).

He et al. [2024] demonstrated that GPT-3.5-Turbo performance varies by up to 40% on code translation tasks depending on prompt template, with GPT-4-32k showing 300% improvement when switching from JSON to plain text on HumanEval. Optimal format choices between models within the same family showed IoU scores below 0.2, indicating minimal overlap.

In the code domain, Chen et al. [2024] catalogued 20 types of prompt perturbations (additions, deletions, edits, swaps, paraphrases) and measured degradation up to 21.2% on code generation benchmarks, with average degradation of 4.8–6.1% across perturbation types.

2.3 Content-Format Interdependence

Liu et al. [2025] provided the first formal treatment of content-format integrated optimization (CFPO). Through ablation studies on GSM8K with LLaMA-3.1-8B, they demonstrated a 1.44 percentage point gap between sequential optimization (content then format: 61.94%) and joint optimization (63.38%), providing direct evidence that format effectiveness depends on content and vice versa. CFPO achieved gains of 4.69–31.60 percentage points across tasks

and models, with pre-trained models showing greater format sensitivity than instruction-tuned models.

Ngweta et al. [2025] proposed Mixture of Formats (MOF), which diversifies formatting styles across few-shot examples within a single prompt. MOF achieved a 46% reduction in style-induced brittleness on Llama-2-13b. However, MOF addresses within-model brittleness; it does not solve cross-model format adaptation.

2.4 Cross-Model Prompt Transfer

Wang et al. [2025] introduced PromptBridge, a training-free framework addressing what they term “model drifting”—the performance degradation when reusing prompts across models. Their approach, Model-Adaptive Reflective Prompt Evolution (MAP-RPE), uses 54 alignment tasks for calibration, maintaining multiple prompt populations across evolutionary islands. PromptBridge achieved relative improvements of 27.39% on SWE-Bench Verified and 46.54% on CodeContests when transferring from GPT-4o to target models including o3, Llama-3.1-70B, and Claude variants.

PromptBridge is the closest related work to our approach. Key differences: PromptBridge learns mappings through calibration requiring high-capability LLMs and significant compute; our approach uses declarative model profiles with deterministic structural transforms and optional lightweight LLM adaptation. PromptBridge operates as a research framework; `promptc` operates as a transparent HTTP proxy requiring zero code changes to existing tools.

2.5 Prompt Standardization and Continuous Prompts

Several efforts have addressed prompt portability from different angles. PromptSource [Bach et al., 2022] provides a repository of over 2,000 standardized prompt templates for approximately 170 datasets, enabling template reuse across models. However, PromptSource standardizes template *content*; it documents but does not solve model-specific performance variations arising from identical templates. HELM [Liang et al., 2022] standardized evaluation across 30 language models and revealed significant format sensitivity—OPT-175B accuracy

ranges from 30.2% to 79.1% on HellaSwag depending on prompt format—reinforcing the portability challenge. OpenPrompt [Ding et al., 2022] provides a modular framework for prompt-learning across multiple architectures (MLM, autoregressive, seq2seq), and its requirement for architecture-specific Template and Verbalizer classes itself highlights the format portability problem.

An alternative paradigm bypasses discrete prompt formatting entirely. Prompt tuning [Lester et al., 2021] learns continuous “soft prompts” (task-specific embedding vectors) through backpropagation, and prefix-tuning [Li and Liang, 2021] optimizes continuous vectors prepended to the input. Both achieve strong performance but produce representations that are inherently model-specific and non-transferable: the learned embeddings are tied to a specific model’s embedding space and cannot be interpreted as human-readable text. These methods represent the extreme end of content coupling in our taxonomy—maximally performant on one model, zero portability to another. Continuous methods trade portability for performance, and in contexts where per-model specialization is acceptable, they may reduce the practical demand for cross-model prompt transfer. Our work focuses on the discrete prompt regime where human readability, auditability, and cross-model transfer are requirements—the regime in which most production agentic tools (Aider, Claude Code, Cursor) currently operate.

2.6 Prompt Optimization Tools

We surveyed 10+ tools across three categories to assess the state of format-level cross-model compilation.

Content optimizers. DSPy [Khatab et al., 2023] compiles declarative language model calls into optimized pipelines. Its adapters use format-agnostic markers (`[[ ## field_name ## ]]`) that work “universally across all language models”—explicitly not model-specific formatting. Promptomatrix (Salesforce) uses DSPy as a backend for content-level optimization.

Output enforcers. Microsoft Guidance constrains generation via interleaved control and logical structure. Outlines (dottxt) enforces output schema compliance via constrained decoding. LMQL com-

piles for execution efficiency and constraint satisfaction. All three control model *outputs*; none adapt *input* prompt formatting per model.

Management platforms. PromptLayer, Braintrust, Humanloop, and Maxim AI provide version control, A/B testing, collaboration, and observability. PromptLayer explicitly advertises “one prompt template for every model.” None perform format-level compilation.

Emerging. MLflow recently added an experimental `optimize_prompts()` API for model switching, validating the market need but not yet production-ready.

Microsoft Prompty defines a portable `.prompty` file format with YAML frontmatter. It handles portability at the template and configuration level (swapping model parameters), not at the format-rewriting level.

Table 1 summarizes the gap analysis. We define three capability dimensions: **Content Opt.** (optimizes *what* the prompt says—instruction selection, example choice, phrasing); **Output Ctrl.** (constrains or structures model *output* format—JSON schemas, grammar enforcement); **Format Comp.** (adapts *input* prompt structure per target model—XML vs. Markdown, section ordering, delimiter conventions). Assessments are based on documentation review and, where available, source code inspection.²

3 Prompt Coupling

3.1 Conceptual Framework

We define **prompt coupling** as the degree of interdependence between a prompt P and a target language model M , where changes to M (or substitution of M with M') require corresponding changes to P to preserve behavioral fidelity. This is a conceptual definition intended to unify fragmented terminology and connect to software engineering theory, not a mathematical formalization with a measurable metric—though we note that Sclar et al. [2024]’s Format-

Spread provides one possible operationalization for the format dimension.

The framework has three key properties:

1. **Continuum.** Coupling exists on a spectrum from tight (highly model-specific) to loose (model-agnostic), not as a binary.
2. **Bidirectional.** P depends on M ’s behavioral grammar (Section 3.2); M ’s performance depends on P ’s structural alignment with that grammar.
3. **Multi-layered.** Coupling operates at format, content, and behavioral levels simultaneously.

3.2 Behavioral Grammar

We introduce the term **behavioral grammar** as a descriptive metaphor for the implicit set of formatting conventions, structural patterns, and interaction styles that a specific LLM has learned to parse most effectively during training. We use “grammar” by analogy to formal language theory—these are parsing preferences, not formally specified rules—and acknowledge that the concept conflates several distinct phenomena: training data distributions, RLHF alignment preferences, system prompt conventions, and tokenizer artifacts. Despite this conflation, the metaphor is useful because it captures *why* the same semantic content performs differently under different structural presentations. Each model develops a distinct behavioral grammar:

- **Claude** models respond preferentially to XML tags (`<instructions>`, `<thinking>`), nested structural markup, and role assignments in system messages. Anecdotal practitioner reports suggest measurably better constraint adherence with XML formatting compared to Markdown, consistent with Anthropic’s own documentation recommending XML structure for complex prompts.
- **GPT-4** models favor Markdown with headers and bold emphasis. GPT-3.5-Turbo shows up to 40% performance variation based on template format, preferring JSON in some tasks [He et al., 2024].
- **Open-source models** exhibit pronounced format sensitivity. LLaMA-2-13B shows 78.3pp variation [Sclar et al., 2024]. Position bias further compounds format sensitivity: different

²Rubric: ✓ = the tool’s documentation explicitly claims this capability *and* we verified it in code or usage; “Partial” = partial support documented but not the primary design goal; “Experimental” = capability announced but not production-ready per the tool’s own documentation; ✕ = no evidence in documentation or source code.

Table 1: Format-level compilation capability across prompt optimization tools. No existing tool performs cross-model format adaptation.

Tool	Content Opt.	Output Ctrl.	Format Comp.	Gap
DSPy 3.1.3	✓	Partial	×	Universal markers, not model-specific
Prompty	×	×	×	Asset packaging only
Guidance	×	✓	×	Output-focused
Outlines	×	✓	×	Output-focused
LMQL	×	✓	×	Runtime compiler, not format compiler
Promptomatix	✓	×	×	Content via DSPy backend
PromptLayer	×	×	×	“One template for every model”
Braintrust	×	×	×	Version control and testing
Humanloop	×	×	×	Collaboration and evaluation
Maxim AI	×	×	×	Observability platform
MLflow	Experimental	×	×	Validates gap, not production-ready

model families exhibit different preferences for where key context appears within the prompt, with no universal ordering strategy.

Behavioral grammar explains *why* prompt coupling exists: models trained on different data distributions learn different structural parsing preferences. The term connects to formal language theory (grammars, parsing rules) while emphasizing that these preferences are learned and implicit rather than formally specified.

3.3 Coupling Taxonomy

We adapt Constantine’s five classical coupling types [Yourdon and Constantine, 1979] to the prompt-model relationship. Three map directly; two require qualification.

Content coupling (tightest). The prompt relies on model-internal formatting quirks: XML tags for Claude, JSON delimiters for GPT-3.5, specific whitespace patterns. Changing the target model requires rewriting the prompt’s structural markup. In software engineering, content coupling means one module depends on another’s internal implementation. The prompt-model analogy is imperfect—a prompt using XML for Claude is exploiting a learned preference, not modifying internals—but the practical consequence is identical: tight, fragile dependence on implementation details.

Common coupling. The prompt assumes shared instruction-following conventions that vary

between model families: chain-of-thought triggers (“Let’s think step by step” vs. `<thinking>` tags), role-setting patterns, or few-shot example formatting. This corresponds to modules sharing implicit global conventions.

Control coupling. The prompt uses structural signals to influence the model’s execution flow—e.g., numbered step sequences that assume a specific reasoning pattern, or explicit mode-switching tokens. This maps to control coupling where one module passes control flags to another. In practice, control coupling in prompts is difficult to isolate from content coupling, as format and flow control are often intertwined.

Stamp coupling. The prompt shares a composite data structure (e.g., a JSON schema, a multi-field template) with the model, where both parties must agree on the schema’s structure. This maps cleanly to stamp coupling in software. Many production prompts exhibit stamp coupling through tool-use schemas and structured output specifications.

Data coupling (loosest). The prompt communicates only through a task description and input/output specification, making no assumptions about the model’s format preferences. This is the most portable form but may sacrifice performance on any individual model.

We focus our analysis primarily on content, common, and data coupling because they are most directly observable in production systems and most amenable to format-level compilation. Control and

stamp coupling, while present, tend to be addressed by existing tools (Guidance, Outlines) that enforce output schemas.

The taxonomy provides a prescriptive design principle: **minimize prompt coupling** where portability matters, accepting that tighter coupling may yield better performance on a specific target. This directly parallels the software engineering principle of loose coupling for maintainability.

3.4 The Intent-Output Pipeline

To analyze where coupling enters LLM-based systems, we propose the **intent-output pipeline**: a four-stage model of the transformation from developer intent to system output.

1. **Encoding.** The developer translates intent into a prompt. This is lossy: the developer has learned model-specific phrasing patterns (“Claude-speak” or “GPT-speak”), introducing coupling at the human level.
2. **Interpretation.** The model applies its behavioral grammar to parse the prompt. Structural alignment between the prompt format and the model’s grammar determines interpretation fidelity.
3. **System wrapping.** Agentic tools add system prompts, tool schemas, skill definitions, and memory context—all tuned for a specific model. This layer multiplies coupling beyond the user’s prompt.
4. **Execution.** The model generates output. Quality depends on cumulative alignment across all prior stages.

Each stage degrades signal through coupling-induced loss. Agentic coding tools—Claude Code, Cursor, Aider—are significant because they introduce coupling at *all four stages simultaneously*: the developer’s muscle memory (stage 1), the model’s format preferences (stage 2), the tool’s system prompts and schemas (stage 3), and the model’s code generation behavior (stage 4).

4 Coupling in Production Agentic Tools

We examined three production agentic coding tools to characterize the nature and extent of prompt coupling in deployed systems.

4.1 Aider: Quantifying Format-Level Dependencies

Aider³ [Aider-AI, 2024] maintains a 2,718-line model configuration file (`model-settings.yml`) containing 313 distinct model configurations with 0–11 format-specific parameters each (median: 5).

Edit format divergence. Three distinct edit instruction formats are assigned per model: `diff`, `udiff`, and `editor-diff`. GPT-4-turbo uses `udiff`; GPT-4o uses `diff`; Claude-3-opus uses `diff`. The same semantic task (code editing) requires structurally different instructions per model.

Opposite behavioral prompts. Most models are configured with `lazy: true`, which injects the system prompt fragment “You NEVER leave comments describing code without implementing it!” Claude-3-7-sonnet is configured with `overeager: true`, injecting the opposite: “Do what they ask, but no more. Do not improve...unrelated parts.” The same tool must instruct different models in contradictory ways to achieve the same behavioral outcome.

Format workarounds. Over 30 model variants require the system prompt prefix “Formatting re-enabled.” to function correctly—a workaround for reasoning models that otherwise suppress structured output. The parameter `examples_as_sys_msg` varies even within the same model family across versions (GPT-4o-2024-08-06: `true`; earlier variants: `false`).

Aider’s configuration file constitutes direct evidence that format-level coupling is not theoretical—it is a production engineering problem requiring per-model maintenance.

³<https://github.com/Aider-AI/aider>, commit b9050e1, accessed February 18, 2026. The configuration count reflects rapid growth in supported models; earlier commits contain fewer entries (e.g., ~134 in mid-2025), illustrating the ongoing maintenance burden.

4.2 Claude Code: Architectural Coupling

Claude Code operates exclusively with Anthropic models by default. Its system prompt contains 18 built-in tool descriptions using a proprietary naming convention (`mcp__servername__toolname`). The prompt structure, tool schemas, and multi-step instruction patterns are engineered for Claude’s behavioral grammar.

Community-built workarounds illustrate the coupling friction: LiteLLM proxy (translates API schema), Ollama compatibility layer (emulates Anthropic Messages API), and “Claude Bridge” (patches Node.js `fetch` to intercept and transform requests). Several LLM vendors (Moonshot K2, Qwen-Coder) have begun supporting Claude’s API interface specifically to capture Claude Code’s user base—a market response driven by coupling.

4.3 Cursor: Behavioral Coupling

Cursor supports model selection but documents systematic behavioral differences. Models are categorized as “inferring intent” (O3, Claude-4-Sonnet, Gemini-2.5-Pro) versus “requiring explicit instructions”—a behavioral grammar distinction that affects prompting strategy. The official recommendation when prompts underperform: “switch to a different model and try again.” Mid-conversation model switching is supported precisely because per-model prompt compatibility is a recognized user pain point.

4.4 Content Standardization Without Format Adaptation

The AGENTS.md specification, adopted by OpenAI, Google, Cursor, Factory, and widely deployed across public repositories, standardizes the *content* of instructions given to agentic tools. It does not address *format*: how those instructions should be structurally presented to different models. This gap—standardized content, unstandardized format—is precisely where prompt coupling operates.

5 promptc: Format-Level Prompt Compilation

To demonstrate that format-level prompt compilation is feasible, we developed `promptc`, an open-source transparent HTTP proxy.⁴

5.1 Design Principles

1. **Zero integration cost.** `promptc` operates as a transparent HTTP proxy. Existing tools (Claude Code, Cursor, Aider) require only a base URL change (`ANTHROPIC_BASE_URL=http://localhost:4000`).
2. **Deterministic where possible.** Structural transforms (XML ↔ Markdown, section reordering) are deterministic, avoiding LLM dependency for basic reformatting.
3. **Semantic adaptation when needed.** An optional second pass uses a local LLM (via Ollama) for deeper intent-preserving adaptation.
4. **Community-maintainable profiles.** Model behavioral grammars are encoded as YAML profiles, updatable independently of the proxy code.

5.2 Architecture

`promptc` intercepts HTTP requests to `/v1/*` endpoints, detecting the API format (Anthropic or OpenAI schema) and target model. It applies a two-pass rewriting pipeline:

Pass 1: Deterministic structural transform. Based on the target model’s profile, the proxy performs format conversion (e.g., XML tags to Markdown headers or vice versa), section reordering according to the model’s preferred information position, and delimiter adaptation.

Pass 2: LLM-powered semantic adaptation (optional). For prompts requiring deeper changes, a local Ollama model (default: Qwen-2.5-Coder-14B) rewrites the prompt to match the target model’s behavioral grammar while preserving all original instructions and intent.

Additional components include session tracking (detects context gaps in multi-turn conversations

⁴Source code: <https://github.com/shakecodeslikecray/promptc>

and enriches prompts with recent history), SQLite-backed caching (avoids redundant rewrites via hash-based lookup), and a web dashboard for monitoring rewrite activity.

5.3 Illustrative Transform Examples

To make the compilation process concrete, we show three worked examples of `promptc`’s deterministic transforms. These are representative of the structural rewrites the tool performs; they are illustrative, not benchmarked for downstream task performance.

Example 1: XML to Markdown (Claude → GPT-4o). A prompt authored for Claude using XML structure:

```
<instructions>
  <role>You are a code reviewer.</role>
  <task>Review the following function for
    bugs and security issues.</task>
  <constraints>
    <item>Be concise</item>
    <item>Cite line numbers</item>
  </constraints>
</instructions>
```

is transformed to Markdown for GPT-4o:

```
## Instructions

**Role:** You are a code reviewer.

**Task:** Review the following function for
bugs and security issues.

**Constraints:**
- Be concise
- Cite line numbers
```

The semantic content is identical; the structural presentation matches each model’s preferred formatting conventions.

Example 2: Markdown to XML (GPT-4o → Claude). The reverse direction applies symmetrically. A Markdown-formatted system prompt with headers and bullet lists is converted to nested XML tags with `<item>` elements.

Example 3: Section reordering. Some models perform better when constraints appear before the task description; others prefer

task-first ordering. Given a profile specifying `constraint_position: before_task`, `promptc` reorders sections accordingly without altering their content.

These transforms are deterministic and fast (sub-millisecond). The optional Pass 2 (semantic adaptation) handles cases that structural transforms cannot address, such as rephrasing chain-of-thought triggers (“Let’s think step by step” → `<thinking>` tags) or adjusting persona framing conventions.

Limitations of these examples. We emphasize that showing input/output pairs demonstrates *feasibility*, not *efficacy*. Whether these transforms preserve or improve downstream task performance is an empirical question we leave to future work. The transforms may introduce subtle semantic drift, particularly for prompts with ambiguous structure, and the section reordering heuristic assumes that model preferences are accurately captured in profiles.

5.4 Model Profiles

Each model profile is a YAML file specifying format preferences, known strengths, and weaknesses:

```
# profiles/claude-sonnet.yaml
model: claude-sonnet-4
format_preferences:
  structure: xml_tags
  emphasis: nested_xml
  persona_placement: system_message
  cot_style: "<thinking> tags"
  example_format: xml_wrapped
known_strengths:
  - long_context
  - instruction_following
  - xml_parsing
known_weaknesses:
  - refusal_sensitivity
```

Profiles serve as declarative encodings of behavioral grammars, separating the *knowledge* of model preferences from the *mechanism* of transformation. New models require only a new profile, not code changes.

Table 2: Comparison of cross-model prompt adaptation approaches.

	PromptBridge		promptc
Method	Calibration	+	Profile-based deterministic + semantic transforms
Requires	Calibration tasks, high-capability LLM		YAML profiles, optional local Ollama
Integration	Research framework		Transparent HTTP proxy
Compute	Heavy (island evolution)		Light to moderate
Updates	Re-calibrate per model		Update YAML profile
Scope	Prompt-level transfer		Format compilation + session awareness

5.5 Comparison with Related Approaches

Table 2 positions `promptc` relative to existing approaches.

The relationship between DSPy and `promptc` is complementary, not competitive. DSPy optimizes prompt *content* (which instructions, which examples). `promptc` optimizes prompt *format* (how those instructions are structurally presented). In compiler terms: DSPy is an algorithm optimizer; `promptc` is a cross-compilation backend targeting different instruction set architectures.

6 Discussion

6.1 Economic Implications

LLM token pricing spans orders of magnitude across providers—from \$0.075 per million input tokens (Gemini 2.0 Flash-Lite, as of February 2026) to \$15.00 (Claude Opus 4), a 200x range—creating strong economic incentive to switch models based on task requirements. As multi-model deployment strategies become more common, prompt coupling emerges as an invisible switching cost. Unlike infrastructure dependencies (visible in deployment configurations) or API dependencies (visible in code), prompt coupling hides in the structural formatting of natural language text. Organizations discover their coupling exposure only when attempt-

ing to switch providers—by which point teams have accumulated months of model-specific prompt engineering expertise that does not transfer.

6.2 Regulatory Context

The UK Competition and Markets Authority identified AI platform lock-in as a regulatory concern in its 2024 AI Foundation Models report [Competition and Markets Authority, 2024]. The CMA’s “Choice” principle mandates that “consumers and businesses should be able to switch and/or use multiple foundation models as necessary without undue difficulty.” The EU AI Act [European Parliament and Council, 2024], with GPAI obligations effective August 2025, establishes transparency requirements that may indirectly support portability.

Prompt coupling may complicate compliance with these regulatory goals. If prompts cannot be meaningfully transferred between models without degradation, the practical ability to switch providers is constrained regardless of API-level portability. Format-level compilation tools like `promptc` could serve as enabling infrastructure in this context, though the gap between format-level portability and the broader switching concerns addressed by regulators should be acknowledged.

6.3 Limitations and Risks

Model convergence. Larger, more capable models may become increasingly format-robust, reducing the practical impact of coupling. However, the trend toward model fragmentation (GLM, DeepSeek, Qwen, Mistral, Llama) suggests the ecosystem is diversifying, not converging. The wide pricing range (200x between cheapest and most expensive models as of early 2026) further incentivizes the use of smaller, more format-sensitive models for appropriate tasks.

Profile maintenance. Model profiles require updating as models evolve. We mitigate this through community-maintainable YAML profiles (analogous to browser compatibility tables like caniuse.com), but profile staleness remains a practical concern.

Evaluation methodology. Measuring format-attributable performance deltas in isolation is methodologically challenging, as Liu et al. [2025]

demonstrated with their content-format interdependence findings. We encourage the community to develop standardized benchmarks for cross-model prompt portability, as Wang et al. [2025] also recommend.

Latency. The two-pass rewriting pipeline adds latency. Deterministic transforms are fast; the optional Ollama pass introduces variable delay. Caching mitigates repeat rewrites. We defer detailed latency characterization to future work.

Scope of adaptation. Format-level transforms may not capture deeper behavioral differences between models (e.g., reasoning patterns, refusal boundaries). `promptc` addresses the structural layer; deeper adaptation may require approaches like PromptBridge’s calibration-based transfer.

6.4 The Compiler Analogy

The relationship between prompt tools and traditional compilers clarifies the landscape:

- **DSPy** $\approx$ algorithm optimizer (improves program logic)
- **PromptBridge** $\approx$ profile-guided optimization (uses runtime data to improve compilation)
- **promptc** $\approx$ cross-compilation backend (targets different ISAs from the same source)
- **Guidance/Outlines** $\approx$ output format specification (like printf format strings)

The field has optimizers for prompt *semantics* but not prompt *syntax*. There is no “format compiler” equivalent to how GCC or LLVM optimize code for different CPU architectures while preserving program semantics.

7 Conclusion

We have introduced **prompt coupling** as a conceptual framework for the degree of interdependence between prompts and their target language models. The term fills a gap in a fragmented terminology landscape, connecting LLM-specific observations to 50 years of software engineering theory on coupling, modularity, and design.

Our systematic observation of coupling patterns in production agentic tools—Aider (313 model configurations), Claude Code (architectural

model dependency), and Cursor (behavioral model assumptions)—demonstrates that prompt coupling is not a theoretical concern but a systematic production engineering problem. Our survey of prompt optimization tools confirms that format-level cross-model compilation remains unaddressed.

`promptc` demonstrates the feasibility of a format-level prompt compiler operating as a transparent HTTP proxy. We release it as open-source software and invite the community to contribute model profiles, evaluate format-attributable performance deltas, and extend the approach. A key direction for future work is empirical evaluation: benchmarking task performance with and without format compilation, measuring latency overhead, and ablating the contribution of deterministic transforms versus semantic adaptation.

As the LLM ecosystem fragments across providers and model families, and as multi-model strategies become the enterprise norm, prompt coupling will increasingly determine the practical portability of AI-powered systems. We hope this framework—and the design principle of minimizing prompt coupling—proves as durable as the software engineering concept it extends.

Acknowledgments

The author thanks the anonymous reviewers for their feedback. The `promptc` implementation and this paper were developed independently; the author discloses that `promptc` is his own open-source project, and the tool evaluation in this paper should be read with that context.

References

- Aider-AI. Aider: Ai pair programming in your terminal, 2024. URL <https://github.com/Aider-AI/aider>.
- Stephen H. Bach, Victor Sanh, Zheng Xin Yong, Albert Webson, Colin Raffel, Nihal V. Nayak, Abheesht Sharma, Taewoon Kim, M. Saiful Bari, Thibault Févry, et al. PromptSource: An integrated development environment and repository for natural language prompts. In *Proceedings of*

- the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 93–104. Association for Computational Linguistics, 2022.
- Junkai Chen et al. Nlperturbator: Studying the robustness of code llms to natural language variations in prompts. *arXiv preprint arXiv:2406.19783*, 2024.
- Competition and Markets Authority. Ai foundation models: Technical update report. Technical report, UK Government, April 2024. URL <https://www.gov.uk/cma-cases/ai-foundation-models>.
- Ning Ding, Shengding Hu, Weilin Zhao, Yulin Chen, Zhiyuan Liu, Haitao Zheng, and Maosong Sun. OpenPrompt: An open-source framework for prompt-learning. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 105–113. Association for Computational Linguistics, 2022.
- European Parliament and Council. Regulation (eu) 2024/1689 (ai act), 2024. Entered into force August 1, 2024.
- Jia He, Mukund Rungta, David Koleczek, Arshdeep Sekhon, Franklin X Wang, and Sadid Hasan. Does prompt formatting have any impact on llm performance? *arXiv preprint arXiv:2411.10541*, 2024.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Mober, et al. Dspy: Compiling declarative language model calls into state-of-the-art pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059. Association for Computational Linguistics, 2021.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4582–4597. Association for Computational Linguistics, 2021.
- Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022.
- Yuanye Liu, Jiahang Xu, Li Lyna Zhang, Qi Chen, Xuan Feng, Yang Chen, Zhongxin Guo, Yuqing Yang, and Peng Cheng. Beyond prompt content: Enhancing llm performance via content-format integrated prompt optimization. *arXiv preprint arXiv:2502.04295*, 2025.
- Lilian Ngweta, Kiran Kate, Jason Tsay, and Yara Rizk. Towards llms robustness to changes in prompt format styles. In *NAACL Student Research Workshop*, 2025. arXiv:2504.06969.
- Amirhossein Razavi, Mina Soltangheis, Negar Arabzadeh, Sara Salamat, Morteza Zihayat, and Ebrahim Bagheri. Benchmarking prompt sensitivity in large language models. *arXiv preprint arXiv:2502.06065*, 2025.
- Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11324.
- Wayne P. Stevens, Glenford J. Myers, and Larry L. Constantine. Structured design. *IBM Systems Journal*, 13(2):115–139, 1974.
- Yaxuan Wang, Quan Liu, Zhenting Wang, Zichao Li, Wei Wei, Yang Liu, and Yujia Bao. Prompt-bridge: Cross-model prompt transfer for large language models. *arXiv preprint arXiv:2512.01420*, 2025.
- Edward Yourdon and Larry L. Constantine. *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design*. Prentice-Hall, 1979.